Computational Logic

Other deductive systems

Sandra Alves - DCC/FCUP

2026/27

Previously on LC...

We have define:

- Gentzen's natural deduction systems.
- No axioms assumed
- Rules for elimination and introduction of connectives.

Hilbert deductive systems, H

Initially used in attempts to mechanize mathematical proofs (19th century and early 20th century) (also by G. Frege and B. Russell)

Assuming only the complete set of connectives $\{\neg, \rightarrow\}$, it takes the form:

Axioms

- $\phi \rightarrow (\psi \rightarrow \phi)$
- $(\phi \rightarrow (\psi \rightarrow \theta)) \rightarrow ((\phi \rightarrow \psi) \rightarrow (\phi \rightarrow \theta))$
- $(\neg\psi \rightarrow \neg\phi) \rightarrow ((\neg\psi \rightarrow \phi) \rightarrow \psi)$

Inference rules

- *Modus ponens*: from ϕ and $\phi \rightarrow \psi$, infer ψ

Hilbert deductive systems, H

Theorem (of deduction): If $\Sigma \cup \{\psi\} \vdash_H \theta$ then $\Sigma \vdash_H \psi \to \theta$.

Theorem: $\Sigma \vdash_{DN} \phi$ if and only if $\Sigma \vdash_H \phi$.

Proof:

($\Leftarrow$) Just see that the axioms of H are theorems of DN. The inference rule matches the DN implication elimination rule ($\to$ **E**)

($\Rightarrow$) It is possible to transform a deduction in DN into a deduction in H. (We won't do that in this course...)]

Corollary: The H deduction system for propositional logic is sound and complete. (As a consequence of soundness and completeness of DN to be proved in the next lectures.)

Automated deductive systems

Even in the *DN* system it is not easy to build an algorithm to determine whether a formula is a theorem, unless you use the completeness (and construct the deduction from the truth table).

But there are other deduction systems that can be automated and in which semantics is not necessary to determine whether a formula ϕ is a theorem.

The essential property of these systems is that, starting from the formula that is intended to be deduced (i.e., considering the deduction from the conclusion to the premises), at each step of a deduction, the formulas are sub-formulas of some formula in a "previous" step.

It is easy to see that the *Modus ponens* rule does not fulfil this property!

Some of these systems

- **Gentzen's *Sequent Calculus***: a "variant" of the natural deduction system.
- **Tableaux**: derives that the formula $\neg\phi$ is a contradiction.
- **Resolution**: also by contradiction, but needs $\neg\phi$ in conjuntive normal form... that is, as sets of clauses.

Smullyan's uniform notation

Propositional formulas are grouped into two categories:

conjunctive α, to which it associates two components α_1 and α_2
disjunctive β, to which it associates two components β_1 or β_2

α	α_1	α_2	β	β_1	β_2
$\phi \wedge \psi$	ϕ	ψ	$\neg(\phi \wedge \psi)$	$\neg\phi$	$\neg\psi$
$\neg(\phi \vee \psi)$	$\neg\phi$	$\neg\psi$	$\phi \vee \psi$	ϕ	ψ
$\neg(\phi \to \psi)$	ϕ	$\neg\psi$	$\phi \to \psi$	$\neg\phi$	ψ

For all variable assignments v, and all formulas α and β:

$$\begin{aligned} v(\alpha) &= v(\alpha_1) \wedge v(\alpha_2) \\ v(\beta) &= v(\beta_1) \vee v(\beta_2). \end{aligned}$$

Tableaux semantics

To derive a formula ϕ, we start with $\neg\phi$ with the purpose of reaching a a contradiction. A deduction is a tree in which the nodes are labeled by formulas, with the root of the tree labeled by $\neg\phi$.

At each step, one leaf of the tree expands according to the *Tableaux* expansion rules:

$$\frac{\neg\neg\phi}{\phi} \qquad \frac{\neg\mathbf{F}}{\mathbf{V}} \qquad \frac{\neg\mathbf{V}}{\mathbf{F}} \qquad \frac{\alpha}{\alpha_1 \\ \alpha_2} \qquad \frac{\beta}{\beta_1 \mid \beta_2}$$

Tableaux semantics

Let r be a branch of the tree labeled ψ.

- If ψ is $\neg\neg\phi$, we expand the leaf of this branch with another one node labeled by ϕ.
- Similarly, if ψ is $\neg\mathbf{F}$, add $\mathbf{V}$ or if it is $\neg\mathbf{V}$, add $\mathbf{F}$.
- If ψ is an α, two nodes are added to r, one labeled with α_1 and the other with α_2 (son of α_1).
- If ψ is a β, add to r two children nodes, one labeled by β_1 the other by β_2.

Tableau

A *tableau* T can be defined inductively by:

Let $\{\phi_1, \ldots, \phi_n\}$ be a set of formulas.

1. A *tableau* for $\{\phi_1, \ldots, \phi_n\}$ is a one branch tree:

$$\begin{array}{c} \phi_1 \\ \phi_2 \\ \vdots \\ \phi_n \end{array}$$

2. If T is a *tableau* for $\{\phi_1, \ldots, \phi_n\}$ and $T^\star$ result from T by applying a *tableaux* expansion rule, then $T^\star$ is a *tableau* for $\{\phi_1, \ldots, \phi_n\}$.

Example

A *tableau* for the set $\{p \wedge (\neg q \vee \neg p)\}$ is:

$$p \wedge (\neg q \vee \neg p)$$
$$p$$
$$\neg q \vee \neg p$$
$$\neg q \qquad\qquad \neg p$$

Satisfabiblity and deduction in *Tableaux*

A branch r of a *tableau* is said to be closed if there is a formula ϕ such that ϕ and $\neg\phi$ occur in r.

A *tableau* is said to be closed if all of its branches are closed.

(The *tableau* in the previous example has a closed branch but is not closed.)

A deduction by *tableau* of ϕ is a *tableau* closed to $\{\neg\phi\}$.

The formula ϕ is a theorem if ϕ has a deduction by *tableau*.

A branch r of a *tableau* is satisfiable if the set of formulas that label its nodes is satisfiable.

A *tableau* T is satisfiable if at least one of its branches is satisfiable.

Soundness and Completeness

Proposition: By applying any *tableau* expansion rule to a satisfiable *tableau*, we obtain a satisfiable *tableau*.

Proposition: If there is a closed *tableau* for a set of formulas Γ, then Γ is not satisfiable.

Theorem: If ϕ has a deduction by *tableau*, then ϕ is a tautology.

Proof: A deduction by *tableau* is a closed *tableau* for $\{\neg\phi\}$. Then, by the proposition above, $\{\neg\phi\}$ is not satisfiable , therefore ϕ is a tautology.

We can also show that:

Theorem: If ϕ is a tautology, then ϕ has a deduction by *tableau*.

Examples

Build *tableaux* deductions for the following formulas:

a) $\neg((p \vee q) \wedge (\neg p \wedge \neg q))$

b) $((p \to q) \wedge (q \to r)) \to \neg(\neg r \wedge p)$

Resolution

A deductive system by refutation: to deduce ϕ, we deduce that $\neg\phi$ is a contradiction.

Let Σ be a set of clauses (disjunction of literals), where we represent the empty clause by $\mathbf{F}$.

The deductive system by *resolution* has no axioms and only one **resolution inference rule**:

$$\frac{C \cup \{p\} \qquad C' \cup \{\neg p\}}{C \cup C'}$$

The conclusion $C \cup C'$ is called the resolvent of premises.

A deduction of $\mathbf{F}$ from a set $\mathcal{C}$ of clauses is called a refutation $\mathcal{C}$, and a deduction by resolution of a formula ϕ is a refutation of the clauses that correspond to the FNC of $\neg\phi$.

Example

Given

$$\{\{\neg p\}, \{q, r\}, \{\neg r, \neg s\}, \{p, s\}, \{\neg q, \neg s\}\}$$

we have

$$\dfrac{\dfrac{\{p,s\}\ \{\neg p\}}{\{s\}}\quad \dfrac{\dfrac{\{q,r\}\ \{\neg r,\neg s\}}{\{q,\neg s\}}\ \{\neg q,\neg s\}}{\{\neg s\}}}{\mathsf{F}}$$

Resolution Soundness and Completeness

Soundness: Let $\mathcal{C} = \{C_1, \ldots, C_n\}$ be a non-empty set of clauses of propositional logic.

i. Let C_i and C_j be clauses of $\mathcal{C}$ and R a resolvent of C_i and C_j. Then $\mathcal{C} \models R$.

ii. If $\mathcal{C} \vdash_R \mathbf{F}$, that is, there is a deduction of $\mathbf{F}$ from $\mathcal{C}$ using just the rule of the Resolution, then $\mathcal{C}$ is not satisfiable.

Completeness: If a set of clauses $\mathcal{C}$ is unsatisfiable then there is a deduction $\mathcal{C} \vdash_R \mathbf{F}$.

Exercises

Build deductions by *tableau* and by **resolution** for the following formulas:

a) $(p \to q) \to ((\neg p \to q) \to q)$

b) $q \to (p \to (q \wedge p))$

c) $q \to (\neg p \to \neg(q \to p))$